CHAPTER 1

Recurrent neural networks

The foundational feed-forward architectures established in the preceding chapter operate as static spatial mappings between fixed-dimensional Euclidean vector spaces. In a conventional multi-layer perceptron, an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ is mapped deterministically to an output vector $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$ through a sequence of parameterized affine transformations interleaved with coordinate-wise non-linearities. While this formulation provides universal function approximation under compact domains, its topological contract assumes that every input observation arrives as a self-contained, isolated point of fixed dimensionality, devoid of internal temporal ordering or dynamic duration.

Real-world perceptual and symbolic processes—most notably natural language text, acoustic speech signals, software execution traces, and biological macromolecules—fundamentally violate this static spatial premise. Natural sequences exhibit variable lengths across observations, temporal dependencies where the interpretation of a token depends intrinsically on antecedent context, and translation-invariant motifs that retain identical semantic meaning regardless of the temporal index at which they appear. Applying static feed-forward networks to such sequential signals requires ad-hoc zero-padding, rigid context window truncation, or the redundant allocation of localized parameters across spatial coordinates, inducing severe sample inefficiencies and representational brittleness.

This chapter establishes the theoretical, algorithmic, and architectural foundations of Recurrent Neural Networks (RNNs). We begin by formalizing the structural failure modes of feed-forward networks on sequential data, motivating the necessity of persistent internal memory and position-invariant parameter sharing. We derive the mathematical mechanics of recurrent state transitions, unroll the network across time into an equivalent directed acyclic graph, and formulate the exact gradient accumulation dynamics of Backpropagation Through Time (BPTT). We then analyze the analytical pathologies of vanilla recurrence—specifically the exponential decay and explosion of temporal Jacobians—and examine how gated recurrent architectures, such as the Long Short-Term Memory network and the Gated Recurrent Unit, enforce constant error propagation. Finally, we explore the sequence-to-sequence encoder-decoder paradigm and demonstrate how the fundamental information bottleneck of compressing variable-length sequences into a single static context vector directly motivates the Attention mechanism.

1.1 Limitations of feed-forward architectures on sequential data

Understanding the emergence of recurrent architectures requires analyzing why standard feed-forward networks, despite their universal approximation capabilities, are structurally ill-suited for sequence processing. The limitations of feed-forward networks stem from their rigid dimensional contracts, their localized parameter binding, and their inability to model temporal dynamics without severe compromises.

1.1.1 Fixed dimensionality constraints on inputs and outputs

A standard fully connected neural layer computes an affine transformation followed by an activation function:

$$\mathbf{h} = \sigma\left(\mathbf{W}\mathbf{x} + \mathbf{b}\right), \tag{1.1}$$

where $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ denotes the input vector, $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ represents the weight parameter matrix, and $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$ is the bias vector [**?**]. The algebraic definition of matrix multiplication requires that the column dimension of $\mathbf{W}$ strictly matches the dimensionality of the input vector $\mathbf{x}$. Consequently, any neural network constructed from feed-forward layers imposes an unyielding contract: every input vector must possess exactly d_{in} coordinates, and every output prediction must produce exactly d_{out} values.

In structured machine learning domains, such as tabular databases, this fixed dimensional contract is entirely appropriate because each coordinate of $\mathbf{x}$ corresponds to a specific, predefined feature column. However, unstructured perceptual and symbolic signals are intrinsically variable in extent. In natural language processing, a text sample may span a single-word query, a five-token sentence, or a document containing thousands of words. Similarly, raw acoustic recordings vary in duration from fractions of a second to multiple hours, and computer vision images vary across capture devices in height and width before standardization.

When practitioners attempt to process variable-length sequences using static feed-forward networks, they are forced to define an arbitrary upper bound on sequence length, denoted $T_{\max}$, and enforce dimensional uniformity via artificial zero-padding or heuristic truncation. If an observation sequence has length $T < T_{\max}$, the remaining $T_{\max} - T$ slots are populated with synthetic null tokens; conversely, if $T > T_{\max}$, the sequence is truncated, discarding all information residing beyond the arbitrary horizon. This operational compromise introduces substantial structural deficiencies:

1. **Computational waste**: Zero-padded dimensions must undergo full matrix multiplications throughout all layers, consuming memory bandwidth and floating-point operations on non-informative null values.

2. **Information loss**: Rigid truncation destroys long-range context and truncates syntactic clauses that are essential for accurate semantic inference.

3. **Spurious gradient signals**: Unless complex masking logic is manually applied at every layer, artificial padding tokens contribute synthetic activations that skew batch statistics and destabilize optimization.

The dimensional rigidity of the output layer is equally problematic. In sequential tasks such as syntactic parsing, semantic segmentation, or machine translation, the target output sequence is not a static scalar or fixed-size vector; it is itself an ordered sequence whose length T_{out} varies dynamically and cannot be determined prior to observing the input signal. A feed-forward network with a fixed output dimension d_{out} is mathematically incapable of generating variable-length sequences without enforcing artificial output horizons.

1.1.2 Lack of position invariance and weight localization

The second fundamental pathology of feed-forward networks is their spatial and temporal parameter localization. In a dense layer, each scalar weight W_{ji} is dedicated exclusively to the connection between input coordinate i and hidden neuron j. This architecture binds each parameter to an absolute coordinate index in the input vector space [**?**].

To conceptualize the consequences of parameter localization, consider an image classification task evaluated on flattened pixel vectors. Suppose an image contains the visual motif of a dog located in the upper-left quadrant of the frame. The feed-forward network activates the subset of weights associated with those specific coordinate indices to detect the object. If an identical dog appears in the lower-right quadrant of another image, the corresponding visual features activate an entirely disjoint set of pixel coordinates, corresponding to completely different columns of $\mathbf{W}$. Because the network lacks parameter sharing across spatial positions, it cannot transfer what it learned about dogs in the upper-left corner to dogs appearing in the lower-right corner. The model is forced to independently rediscover and parameterize the visual concept of a dog at every discrete spatial coordinate.

When applied to sequential text processing, this parameter localization leads to severe sample inefficiency. Consider a character-level capitalization task where an input character sequence must be mapped to its correctly cased variant. In natural language, an opening quotation mark " serves as a structural syntactic cue indicating that the immediately following character must be capitalized as the start of a dialogue utterance. In a feed-forward network where a sequence of T characters is concatenated into a single input vector:

$$\mathbf{x} = \left[\mathbf{x}_1^\top, \mathbf{x}_2^\top, \ldots, \mathbf{x}_T^\top\right]^\top \in \mathbb{R}^{T \cdot d_c}, \tag{1.2}$$

the network must learn separate, independent weight parameters to detect quotation marks at position $t = 1$, position $t = 2$, position $t = 3$, and every subsequent temporal offset up to T. If the training corpus contains numerous examples of dialogue starting at index 1 but none starting at index 14, the model will fail to capitalize a dialogue quote appearing at index 14, despite possessing the relevant knowledge at index 1.

Because identical linguistic and acoustic motifs occur at arbitrary temporal offsets, a sequence processing model must satisfy *temporal translation equivariance*: shifting an input pattern in time should produce an identical, equivalently shifted representation without requiring separate parameters. A feed-forward network completely lacks this inductive bias, demanding a parameter budget that scales quadratically with sequence length:

$$N_{\text{parameters}} = O\left(T \cdot d_{\text{in}} \cdot d_{\text{hidden}}\right), \tag{1.3}$$

which becomes computationally intractable for long textual contexts and renders generalization to unseen sequence positions virtually impossible.

1.1.3 The structural requirement for dynamic temporal processing

The failure of feed-forward architectures establishes the formal requirements for an authentic sequential learning paradigm. Mathematically, an ordered sequence is defined as a tuple:

$$\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T), \quad \mathbf{x}_t \in \mathbb{R}^{d_x}, \quad T \in \mathbb{N}^+, \tag{1.4}$$

where the temporal index t specifies chronological or symbolic ordering, and the total length T varies across observations.

To process such sequences effectively, an architecture must satisfy three primary design imperatives:

1. **Length-agnostic ingestion**: The model must operate on sequences of arbitrary length T through an iterative transition operator whose parametric specification is strictly independent of T.

2. **Position-invariant parameter sharing**: The transformation applied at time step t must utilize the exact same parameter tensors as the transformation applied at time step t', enabling structural motifs learned at one temporal position to generalize across all positions.

3. **Contextual persistence and state propagation**: The representation computed at step t must not depend solely on the isolated token $\mathbf{x}_t$, but must integrate a synthesized historical record of all prior observations $(\mathbf{x}_1, \ldots, \mathbf{x}_{t-1})$.

These three requirements directly motivate the architecture of Recurrent Neural Networks, which replace the static, one-shot feed-forward mapping with a dynamical system governed by an internal recurrent state.

1.2 Recurrent neural networks: intuition, unrolling, and cell mechanics

Recurrent Neural Networks address the structural limitations of feed-forward networks by introducing an internal state that persists across temporal steps. Instead of consuming an entire sequence simultaneously, an RNN ingests one token at a time, continuously updating its internal memory to synthesize past observations with current sensory input.

1.2.1 The recurrent intuition and persistent hidden state

The fundamental conceptual mechanism of a recurrent neural network can be understood through the pedagogical analogy of a human reader taking running notes while reading a document. A human reader does not attempt to ingest an entire book in a single visual fixation; rather, the reader scans the text sequentially, word by word. As each new word is read, the reader does not discard previous knowledge, nor do they carry the verbatim text of every preceding page in immediate working memory. Instead, the reader maintains a continuous mental summary—a running piece of paper—recording essential context, active subjects, unresolved clauses, and grammatical state. When a new word is encountered, the reader combines their current mental notes with the incoming word, updates the notes, and optionally generates an interpretation or expectation of upcoming text.

In a Recurrent Neural Network, this mental notepad is formalized as the *hidden state vector*, denoted $\mathbf{h}_t \in \mathbb{R}^{d_h}$. The hidden state serves as an internal, distributed latent representation that encodes the historical trajectory of the sequence up to time step t [**?**]. At each discrete time step $t \in \{1, \dots, T\}$, the recurrent unit operates as an operator characterized by a dual-input, dual-output interface:

$$\text{Inputs at step } t: \quad \mathbf{x}_t \in \mathbb{R}^{d_x} \text{ (current sequence element)}, \quad \mathbf{h}_{t-1} \in \mathbb{R}^{d_h} \text{ (previous state)}, \tag{1.5}$$

$$\text{Outputs at step } t: \quad \hat{\mathbf{y}}_t \in \mathbb{R}^{d_y} \text{ (step prediction)}, \quad \mathbf{h}_t \in \mathbb{R}^{d_h} \text{ (updated state)}. \tag{1.6}$$

Before any sequence elements are observed (at step $t = 0$), the network must initialize its internal memory. In the absence of prior context, the initial state is typically defined as a vector of zeros:

$$\mathbf{h}_0 = \mathbf{0} \in \mathbb{R}^{d_h}, \tag{1.7}$$

representing a clean, unwritten slate. As sequence tokens $\mathbf{x}_1, \mathbf{x}_2, \dots$ arrive, the recurrent transition function modifies $\mathbf{h}_t$ at every step, allowing downstream predictions $\hat{\mathbf{y}}_t$ to be conditioned on the complete historical context $\mathbf{x}_{\leq t}$.

1.2.2 Time unfolding and unrolled computational graphs

The recurrent relationship is traditionally depicted in two equivalent representational forms: the compact *folded* diagram and the explicit *unfolded* computational graph.

In the folded representation (Figure 1.1a), the network is illustrated as a single processing unit possessing a directed cyclic feedback loop: the hidden state output $\mathbf{h}_t$ is fed directly back into the cell to serve as the state input for the subsequent step. While algebraically concise, cyclic graphs cannot be directly evaluated by standard automatic differentiation engines, which require acyclic computation paths to establish reverse topological orderings.

To resolve this cyclic dependency, the recurrent execution is *unfolded in time* (Figure 1.1b). Unfolding expands the recurrent unit across the temporal sequence of length T, replicating the cell into a chain of T interconnected feed-forward units. In this unfolded representation, the cyclic feedback loop is mapped onto a sequence of forward-directed horizontal edges connecting unit $t-1$ to unit t. The resulting structure forms a classical Directed Acyclic Graph (DAG), enabling standard forward inference and reverse-mode automatic differentiation across the temporal chain.

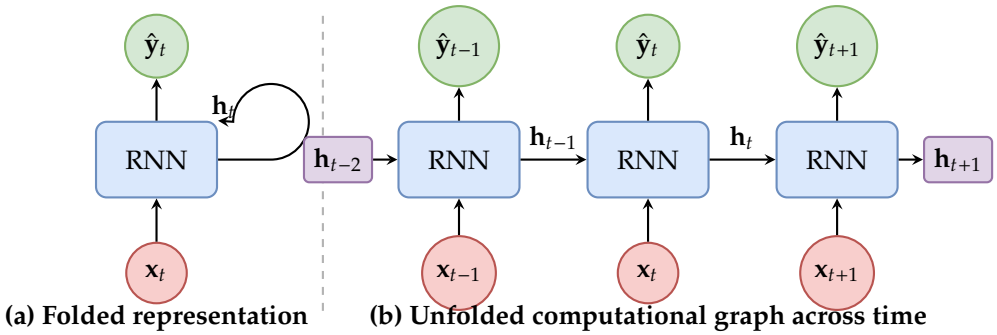


Figure 1.1: Comparison between the compact folded representation of a recurrent neural network with a self-referential temporal feedback loop and its unfolded computational Directed Acyclic Graph (DAG) across consecutive time steps.

To observe how time unfolding coordinates contextual reasoning, consider tracking the character-level casing problem introduced earlier. Suppose the model receives the sequential lowercase stream `['"', 'y', 'o', 'u']`:

1. **Step 1 ($t = 1$)**: The model ingests $\mathbf{x}_1 =$ `'"'` and initial state $\mathbf{h}_0 = \mathbf{0}$. The network emits $\hat{\mathbf{y}}_1 =$ `'"'` and transitions to state $\mathbf{h}_1$. Internally, the parameter transformations configure $\mathbf{h}_1$ to encode the semantic feature: *"an opening quotation mark has occurred; dialogue has begun; the subsequent token must be capitalized"*.

2. **Step 2 ($t = 2$)**: The model ingests $\mathbf{x}_2 =$ `'y'` alongside $\mathbf{h}_1$. Driven by the contextual indicator stored in $\mathbf{h}_1$, the output projection produces an uppercase token $\hat{\mathbf{y}}_2 =$ `'Y'`. Concurrently, the state updates to $\mathbf{h}_2$, encoding: *"the initial dialogue letter has been capitalized; subsequent letters in the current token should revert to lowercase"*.

3. **Step 3 ($t = 3$)**: The model ingests $\mathbf{x}_3 =$ `'o'` alongside $\mathbf{h}_2$. Consulting $\mathbf{h}_2$, the model emits lowercase $\hat{\mathbf{y}}_3 =$ `'o'` and maintains a state reflecting active word continuation.

Through this sequential unfolding, the network resolves local contextual ambiguities dynamically, utilizing the persistent state vector to propagate structural syntactic memory across successive steps.

1.2.3 Internal cell mechanics and state-space transformations

Having examined the external input-output behavior of the recurrent unit, we now formulate the internal algebraic operations governing the Elman recurrent cell [**?**]. The interior of an Elman cell consists of three parameterized affine projections and two non-linear mapping stages.

The mathematical transformation unfolds in four explicit stages:

1. **Input projection**: The input vector $\mathbf{x}_t \in \mathbb{R}^{d_x}$ is projected into the hidden state space via the input-to-hidden parameter matrix $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times d_x}$:

$$\mathbf{u}_t = \mathbf{W}_{xh}\mathbf{x}_t \in \mathbb{R}^{d_h}. \tag{1.8}$$

   This operation aligns the dimensionality of the incoming observation with the hidden representation space, mapping disparate input modalities (such as one-hot character vectors or continuous embeddings) into a standardized coordinate frame.

2. **Recurrent state transformation**: The previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$ is transformed through the hidden-to-hidden recurrent transition matrix $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$:

$$\mathbf{v}_t = \mathbf{W}_{hh}\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}. \tag{1.9}$$

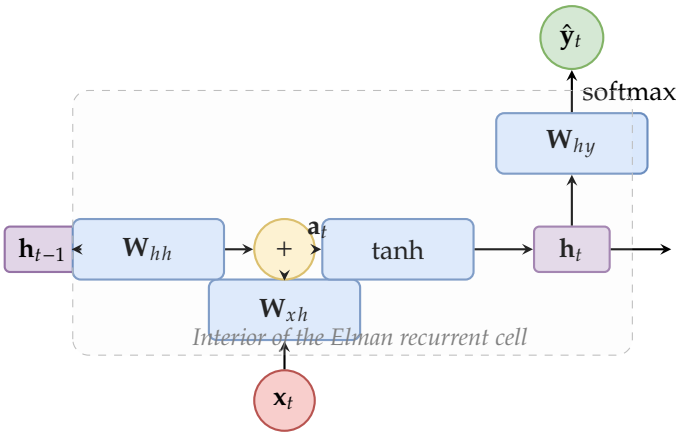


Figure 1.2: Detailed architectural anatomy of the Elman recurrent cell, illustrating the affine projections of the incoming token $\mathbf{x}_t$ and antecedent state $\mathbf{h}_{t-1}$, the linear state aggregation node, the hyperbolic tangent activation function, and the output classification head.

This transformation rotates and scales the historical memory representation, filtering and projecting antecedent information into coordinates compatible with the incoming input.

3. **Linear aggregation and activation**: The projected input $\mathbf{u}_t$ and projected antecedent state $\mathbf{v}_t$ are summed coordinate-wise alongside a learnable hidden bias vector $\mathbf{b}_h \in \mathbb{R}^{d_h}$, yielding the pre-activation state vector $\mathbf{a}_t \in \mathbb{R}^{d_h}$:

$$\mathbf{a}_t = \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h. \tag{1.10}$$

The pre-activation vector is then mapped through an element-wise non-linear activation function, canonically chosen as the hyperbolic tangent (tanh):

$$\mathbf{h}_t = \tanh(\mathbf{a}_t) = \frac{\exp(\mathbf{a}_t) - \exp(-\mathbf{a}_t)}{\exp(\mathbf{a}_t) + \exp(-\mathbf{a}_t)} \in (-1, 1)^{d_h}. \tag{1.11}$$

The hyperbolic tangent squashes state activations into the symmetric bounded interval $(-1, 1)$, preventing unbounded numerical growth across successive temporal steps while centering activations around zero.

4. **Output projection and distribution**: If the task demands a token prediction at time step t, the updated state $\mathbf{h}_t$ is projected into the output target space via the hidden-to-output matrix $\mathbf{W}_{hy} \in \mathbb{R}^{d_y \times d_h}$ and bias $\mathbf{b}_y \in \mathbb{R}^{d_y}$:

$$\mathbf{z}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y \in \mathbb{R}^{d_y}, \tag{1.12}$$

where $\mathbf{z}_t$ represents the unnormalized logit vector. For categorical prediction over a discrete vocabulary $\mathcal{V}$ with dimension $d_y = |\mathcal{V}|$, normalized class probabilities are computed via the softmax operator:

$$\hat{\mathbf{y}}_t = \operatorname{softmax}(\mathbf{z}_t) = \frac{\exp(\mathbf{z}_t)}{\sum_{k=1}^{d_y} \exp(z_{t,k})} \in \Delta^{d_y - 1}. \tag{1.13}$$

To maintain rigorous mathematical clarity, Table 1.1 summarizes the explicit shapes and functional roles of all parameters and representations comprising the Elman recurrent architecture.

Mathematical Symbol	Tensor Shape	Space / Domain	Functional Role
$\mathbf{x}_t$	$d_x \times 1$	$\mathbb{R}^{d_x}$	Input sequence token at step t
$\mathbf{h}_t$	$d_h \times 1$	$(-1, 1)^{d_h}$	Recurrent latent state vector at step t
$\hat{\mathbf{y}}_t$	$d_y \times 1$	$\Delta^{d_y - 1}$	Predicted probability distribution at step t
$\mathbf{W}_{xh}$	$d_h \times d_x$	$\mathbb{R}^{d_h \times d_x}$	Input-to-hidden projection matrix
$\mathbf{W}_{hh}$	$d_h \times d_h$	$\mathbb{R}^{d_h \times d_h}$	Hidden-to-hidden recurrent transition matrix
$\mathbf{W}_{hy}$	$d_y \times d_h$	$\mathbb{R}^{d_y \times d_h}$	Hidden-to-output projection matrix
$\mathbf{b}_h$	$d_h \times 1$	$\mathbb{R}^{d_h}$	Hidden pre-activation bias vector
$\mathbf{b}_y$	$d_y \times 1$	$\mathbb{R}^{d_y}$	Output logit bias vector

Table 1.1: Explicit tensor dimensions, mathematical domains, and functional roles of representations and learnable parameters in the standard Elman recurrent cell.

1.3 Weight sharing and backpropagation through time

The computational elegance and representational power of Recurrent Neural Networks arise from two structural principles: the strict sharing of parameter tensors across all sequence coordinates, and the recursive backpropagation of error signals through the unfolded computational graph.

1.3.1 The weight-sharing paradigm and position invariance

In standard feed-forward networks, processing an input sequence of length T requires distinct weight matrices for each temporal offset, causing parameter count to explode linearly with sequence duration. In sharp contrast, a recurrent neural network enforces *parameter tying*: the exact same weight matrices and bias vectors are reused at every time step:

$$\mathbf{W}_{xh}^{(1)} = \mathbf{W}_{xh}^{(2)} = \cdots = \mathbf{W}_{xh}^{(T)} = \mathbf{W}_{xh}, \quad \mathbf{W}_{hh}^{(1)} = \mathbf{W}_{hh}^{(2)} = \cdots = \mathbf{W}_{hh}^{(T)} = \mathbf{W}_{hh}. \tag{1.14}$$

This architectural invariance yields two decisive advantages:

1. **Position-invariant inference**: Because the transition function is parameterized identically across time, the network applies uniform operational logic to input tokens regardless of their absolute temporal offset. An opening quotation mark encountered at $t = 1$, $t = 25$, or $t = 100$ activates the exact same input projection $\mathbf{W}_{xh}$ and recurrent transition $\mathbf{W}_{hh}$, triggering identical state updates and eliminating the need to independently learn position-specific rules.

2. **Length-invariant parameter complexity**: The total number of learnable parameters in an Elman RNN is:
$$N_{\text{parameters}} = d_h \cdot d_x + d_h^2 + d_h + d_y \cdot d_h + d_y. \tag{1.15}$$
Crucially, Equation 1.15 contains no term depending on sequence length T. The model can process a sequence of 5 tokens or 500 tokens using the exact same parameter budget, enabling deployment on sequences of arbitrary and varying lengths.

1.3.2 Backpropagation through time (BPTT) formulation

Because the computational graph of an RNN unrolls across time into an interconnected chain of operations sharing identical parameters, computing the gradients of the training objective requires a specialized formulation of reverse-mode automatic differentiation termed *Backpropagation Through Time* (BPTT) [**?**].

Consider a training sequence of length T paired with target ground-truth one-hot vectors $\mathbf{y}_1^*, \ldots, \mathbf{y}_T^*$. At each time step t, the network outputs a predicted probability distribution $\hat{\mathbf{y}}_t$ and evaluates a token-level categorical cross-entropy loss:

$$\mathcal{L}_t\left(\hat{\mathbf{y}}_t, \mathbf{y}_t^*\right) = -\sum_{k=1}^{d_y} y_{t,k}^* \log \hat{y}_{t,k}. \tag{1.16}$$

The total scalar sequence loss $\mathcal{L}$ is defined as the temporal summation over all token losses:

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t\left(\hat{\mathbf{y}}_t, \mathbf{y}_t^*\right). \tag{1.17}$$

To update the parameters via gradient descent, we must evaluate the partial derivatives of $\mathcal{L}$ with respect to the shared weight matrices: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hy}}$, $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{xh}}$, and $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}}$.

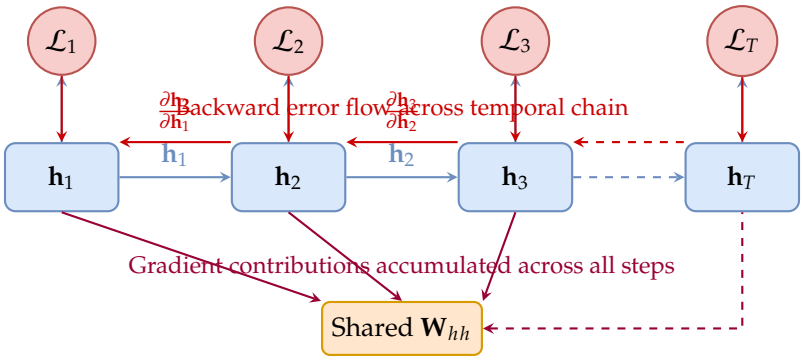


Figure 1.3: The computational flow of Backpropagation Through Time (BPTT). Forward activations propagate chronologically (blue arrows), generating per-step losses. Error signals propagate backward along the temporal chain (red arrows), and local gradient contributions are accumulated into the shared parameter tensor (purple arrows).

The output projection gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hy}}$ depends only on current state activations. By applying the standard cross-entropy softmax gradient derived in Chapter 1:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hy}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_{hy}} = \sum_{t=1}^{T} \left(\hat{\mathbf{y}}_t - \mathbf{y}_t^*\right) \mathbf{h}_t^\top. \tag{1.18}$$

In contrast, evaluating the gradient with respect to the recurrent matrix $\mathbf{W}_{hh}$ requires accounting for the multivariable chain rule across all ancestral steps. Because $\mathbf{W}_{hh}$ was used to compute $\mathbf{h}_t$, which in turn depended on $\mathbf{h}_{t-1}$, which depended on $\mathbf{h}_{t-2}$, a perturbation in $\mathbf{W}_{hh}$ affects the loss $\mathcal{L}_t$ through multiple distinct pathways across time.

Derivation of the recurrent gradient accumulation. To formalize this temporal dependence, let $\frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}$ denote the *direct* partial derivative of $\mathbf{h}_k$ with respect to $\mathbf{W}_{hh}$, treating antecedent state $\mathbf{h}_{k-1}$ as a constant:

$$\frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}} = \operatorname{diag}\left(1 - \mathbf{h}_k^2\right) \otimes \mathbf{h}_{k-1}^\top, \tag{1.19}$$

where $\otimes$ denotes the tensor outer product. The total derivative of step loss $\mathcal{L}_t$ with respect to $\mathbf{W}_{hh}$ sums over all temporal paths from step $k = 1$ up to $k = t$:

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_{hh}} = \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}, \tag{1.20}$$

where $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}$ represents the temporal Jacobian matrix describing how hidden activations at step k propagate forward to influence hidden activations at step t.

Summing over all sequence time steps $t \in \{1, \ldots, T\}$, the complete gradient with respect to $\mathbf{W}_{hh}$ evaluates to:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}. \tag{1.21}$$

By identical reasoning, the gradient with respect to the input projection matrix $\mathbf{W}_{xh}$ accumulates across time as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{xh}} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{xh}}, \quad \text{where} \quad \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{xh}} = \operatorname{diag}\left(1 - \mathbf{h}_k^2\right) \otimes \mathbf{x}_k^\top. \tag{1.22}$$

Equation 1.21 embodies the core operational mechanism of BPTT: during training, the network first performs a forward pass from $t = 1$ to T, computing and caching all states $\mathbf{h}_t$ and

per-step losses $\mathcal{L}_t$. Subsequently, the backward pass propagates error vectors backward through time, accumulating gradient contributions from every historical step into the shared parameter tensors.

1.3.3 Empirical training dynamics: the Sherlock Holmes casing benchmark

To ground the mathematical formulation of BPTT in empirical practice, we analyze the training dynamics of a character-level Elman recurrent network tasked with sentence case correction. The experimental framework directly mirrors the diagnostic benchmark evaluated during course lectures.

Experimental setup. A vanilla Elman RNN is trained on textual excerpts drawn from Arthur Conan Doyle's *The Adventures of Sherlock Holmes*. The input sequence consists of raw sentences converted entirely to lowercase, represented as a stream of 256-dimensional one-hot vectors corresponding to standard ASCII characters ($d_x = 256$). The network utilizes a hidden state dimension $d_h = 32$, initialized with $\mathbf{h}_0 = \mathbf{0}$, and projects to a 256-dimensional categorical output distribution ($d_y = 256$). Optimization is conducted over 10 epochs using the Adam optimizer [**?**] minimizing token-level cross-entropy loss.

The global training loss trajectory exhibits a characteristic progression: starting from an initial loss $\mathcal{L} \approx 3.0$, the loss drops to 2.0 after 2 epochs, reaches 1.72 at epoch 4, drops sharply to 0.95 at epoch 6, and asymptotically plateaus around 0.85 by epoch 10. Inspecting the qualitative outputs generated for a held-out test utterance reveals five distinct developmental stages of sequential pattern acquisition:

$$\text{Input text:} \quad \texttt{"it may be so, mr. holmes"} \tag{1.23}$$

Epoch	Loss $\mathcal{L}$	Predicted Character Sequence Output
0	3.00	`\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x0`
1	2.45	`eeee eeee eeee eeee eeee eeee`
3	1.72	`ie mae be so, mr.  holmes`
5	0.95	`"it may be so, mr.  holmes"`
7	0.88	`"It may be so, mr.  Holmes"`
10	0.84	`"It may be so, mr.  Holmes"`
Target	—	`"It may be so, Mr.  Holmes"`

Table 1.2: Empirical evolution of character-level casing predictions produced by a 32-dimensional Elman RNN across 10 training epochs on an excerpt from *The Adventures of Sherlock Holmes*. The progression illustrates the acquisition of padding bias, unigram frequency priors, whitespace structure, identity copying, and local punctuation-driven capitalization.

Deconstructing the progression recorded in Table 1.2 reveals how gradient descent navigates the sequential hypothesis space:

1. **Epoch 0 (Null-byte padding bias)**: The uninitialized model produces repeated null bytes (`\x00`). In training batches where variable-length strings are aligned using zero-padding, the null character exhibits high empirical prevalence. The optimizer initially exploits this zero-order marginal distribution to achieve trivial loss reduction.

2. **Epoch 1 (Unigram letter frequency prior)**: As backpropagation penalizes null outputs on non-padding tokens, the network discovers the marginal unigram frequency distribution of the English language. It repeatedly predicts the character `'e'`—the most statistically frequent English character—interleaved with periodic whitespace tokens.

3. **Epoch 3 (Whitespace and lexical boundaries)**: The model masters word boundary segmentation. It reliably predicts space characters following character clusters, establishing basic lexical cadence even while specific character identities remain noisy.

4. **Epoch 5 (Identity copying and the steep loss drop)**: The loss experiences a dramatic decline from 1.72 to 0.95. The recurrent network discovers the global structural invariant of the task: *most output characters are identical to their input counterparts*. By configuring the composite transformation $\mathbf{W}_{hy} \tanh(\mathbf{W}_{xh}\mathbf{x}_t) \approx \mathbf{x}_t$, the network implements an identity bypass, echoing the input string verbatim.

5. **Epoch 7 (Local syntactic rule discovery)**: The network learns fine-grained sequential rules. Having observed that opening quotation marks precede dialogue, the state $\mathbf{h}_t$ transitions to capitalize the subsequent character, transforming `"it` into `"It`. Furthermore, it learns that the character sequence `holmes` constitutes a proper noun, correctly capitalizing the `'H'` in `Holmes`.

The causal horizon failure. Despite reaching training convergence by Epoch 10, the model exhibits a persistent failure: it fails to capitalize the title `mr.`, leaving it as lowercase `mr. Holmes`. This failure exposes the fundamental theoretical limitation of strictly causal, one-to-one recurrent processing. When the network ingests the character `'m'` at time step $t = 15$, its decision to output `'M'` or `'m'` must be made strictly on the basis of antecedent observations $\mathbf{x}_{\leq 15}$ = `'"it may be so, '`. At this point in time, the network has not yet observed the upcoming characters `'r'`, `'.'`, or `'holmes'`. In general English prose, words beginning with the letter `'m'` (such as *may, much, more, my, must*) are lowercase in over 95% of occurrences. Because the causal one-to-one RNN is prohibited from observing future tokens, Bayes' optimal decision rule under cross-entropy forces the network to predict lowercase `'m'`.

Resolving this structural ambiguity requires an architecture that can inspect the entire sentence—including upcoming future tokens—before committing to an output classification.

1.4 Pathologies of vanilla RNNs and gated architectures

While the Elman recurrent network resolves the dimensional and parameter-sharing limitations of feed-forward models, it suffers from severe analytical and computational pathologies that impede learning across long temporal spans.

1.4.1 The vanishing and exploding gradient problem

The primary mathematical pathology governing vanilla recurrent architectures is the vanishing and exploding gradient phenomenon [**? ?**]. To understand its origin, we examine the temporal Jacobian matrix connecting hidden state $\mathbf{h}_k$ to hidden state $\mathbf{h}_t$ in Equation 1.21.

Applying the multivariate chain rule across $t-k$ intermediate recurrent steps yields a continuous product of Jacobian matrices:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}. \tag{1.24}$$

Recalling the hidden state transition equation $\mathbf{h}_j = \tanh(\mathbf{W}_{xh}\mathbf{x}_j + \mathbf{W}_{hh}\mathbf{h}_{j-1} + \mathbf{b}_h)$, the single-step temporal Jacobian evaluates to:

$$\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \operatorname{diag}\left(1 - \mathbf{h}_j^2\right) \mathbf{W}_{hh}^\top, \tag{1.25}$$

where $\operatorname{diag}(1-\mathbf{h}_j^2) \in \mathbb{R}^{d_h \times d_h}$ is a diagonal matrix containing the first derivatives of the element-wise hyperbolic tangent function, $\tanh'(a) = 1 - \tanh^2(a)$.

Substituting Equation 1.25 back into the chain product of Equation 1.24 produces:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^{t} \operatorname{diag}\left(1 - \mathbf{h}_j^2\right) \mathbf{W}_{hh}^\top. \tag{1.26}$$

The matrix product in Equation 1.26 contains $t-k$ successive matrix multiplications of the transposed recurrent weight matrix $\mathbf{W}_{hh}^\top$, modulated at each step by the activation derivatives.

Let $\lambda_{\max}$ denote the largest eigenvalue (spectral radius) of the recurrent weight matrix $\mathbf{W}_{hh}$. Because the derivative of hyperbolic tangent is strictly bounded such that $0 < 1 - h_{j,m}^2 \leq 1$, the asymptotic magnitude of the temporal Jacobian is bounded by:

$$\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| \leq \left(\gamma \left\| \mathbf{W}_{hh}^\top \right\|\right)^{t-k}, \quad \text{where } \gamma \leq 1. \tag{1.27}$$

This analytical bound establishes two opposing failure modes:

1. **Vanishing gradients**: If the spectral radius satisfies $\|\mathbf{W}_{hh}\| < 1$, or if hidden activations operate in saturating regimes where $1 - h_j^2 \ll 1$, the product decays exponentially toward zero as temporal distance $t-k$ increases:

$$\lim_{t-k\to\infty} \left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| = 0. \tag{1.28}$$

   To appreciate the speed of this decay, consider a sequence of moderate length where $t-k = 50$, and suppose the effective scaling factor is 0.90. Over 50 time steps, the gradient scaling factor diminishes to:

$$0.90^{50} \approx 0.00515, \tag{1.29}$$

attenuating the error signal by more than two orders of magnitude. For $t - k = 100$, the scaling factor drops to $0.90^{100} \approx 2.65 \times 10^{-5}$. Consequently, error signals computed at time step t exert virtually zero influence on parameter updates at distant antecedent steps $k \ll t$. The network becomes incapable of learning long-range temporal dependencies.

2. **Exploding gradients**: If the spectral radius satisfies $\|\mathbf{W}_{hh}\| > 1$ and activations remain in the linear regime near zero ($\gamma \approx 1$), the matrix product grows exponentially with temporal distance:
$$\lim_{t-k\to\infty} \left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| = \infty. \tag{1.30}$$
Exploding gradients cause parameter updates $\Delta\mathbf{W} = -\eta\nabla_{\mathbf{W}}\mathcal{L}$ to become astronomically large, catapulting parameters across the loss surface into regions of numerical overflow (`NaN`) and destroying previously acquired representations.

While exploding gradients can be mitigated using the heuristic engineering technique of *gradient clipping by global norm* [**?**]:

$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g} & \text{if } \|\mathbf{g}\|_2 \le v, \\ v\frac{\mathbf{g}}{\|\mathbf{g}\|_2} & \text{if } \|\mathbf{g}\|_2 > v, \end{cases} \tag{1.31}$$

where $v > 0$ defines the maximum allowable gradient norm, gradient clipping cannot resolve vanishing gradients. When error gradients vanish to machine precision zero, no simple rescaling can restore the lost directional signal.

1.4.2 Long-term dependency decay and computational sequentiality

The mathematical vanishing of gradients manifests behaviorally as the loss of long-term temporal memory. Because the hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$ has a strictly finite coordinate dimensionality, it must compress everything observed from step 1 to step t into a fixed-size vector. At each step, Equation 1.11 subjects the state to non-linear transformation and vector addition with incoming tokens:

$$\mathbf{h}_t = \tanh\left(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h\right). \tag{1.32}$$

In practice, new observations continuously overwrite the coordinates of $\mathbf{h}_t$. Unless specific architectural constraints prevent overwriting, information concerning tokens observed dozens of steps in the past decays exponentially, leaving the hidden state dominated almost entirely by the most recent 5 to 10 tokens.

To illustrate this failure mode, consider the diagnostic sequence evaluated in lecture:

Prompt: `"The mouse was getting chased by the cat through the big house.` (1.33)

`After a long chase, the cat finally managed to catch the [ ]"` (1.34)

To correctly predict the target noun `"mouse"` at the sequence terminus, a language model must retain the identity of the original grammatical patient (`"mouse"`) across an intervening clause spanning more than 20 tokens. In a standard Elman RNN, the intervening tokens (*"through the big house. After a long chase, the cat finally managed to catch the"*) trigger repeated state transitions that overwrite the coordinates storing `"mouse"`. Consequently, when the model predicts the final token, its state reflects recent terms such as *"catch"*, prompting the generation of semantically plausible but contextually disconnected completions, such as `"ball"` or `"tail"`, while failing to recover the true referent.

Beyond memory degradation, vanilla RNNs suffer from a profound hardware inefficiency: *computational sequentiality*. By definition, evaluating state $\mathbf{h}_t$ requires antecedent state $\mathbf{h}_{t-1}$ as an

explicit mathematical input. Consequently, a recurrent neural network cannot compute step t until step $t-1$ has fully terminated:

$$\mathbf{h}_1 \longrightarrow \mathbf{h}_2 \longrightarrow \mathbf{h}_3 \longrightarrow \cdots \longrightarrow \mathbf{h}_T. \tag{1.35}$$

This temporal serialization creates a strict dependency chain along the sequence dimension, precluding the parallel evaluation of sequence tokens on modern massively parallel hardware accelerators (GPUs and TPUs). While batch instances can be processed in parallel across sequences, training runtime along the sequence dimension scales strictly as $O(T)$, severely limiting the feasibility of scaling vanilla RNNs to long document contexts.

1.4.3 Gated recurrent units: LSTM and GRU

To overcome the vanishing gradient pathology and prevent the continuous overwriting of historical context, Hochreiter and Schmidhuber introduced the *Long Short-Term Memory* (LSTM) network in 1997 [**?**].

The foundational architectural innovation of the LSTM is the decoupling of memory into two distinct operational streams:

1. **Cell state ($\mathbf{c}_t \in \mathbb{R}^{d_h}$)**: Functions as a dedicated, linear long-term memory trunk (colloquially termed the "conveyor belt").
2. **Hidden state ($\mathbf{h}_t \in \mathbb{R}^{d_h}$)**: Serves as the short-term, task-facing output representation used for immediate predictions and gate conditioning.

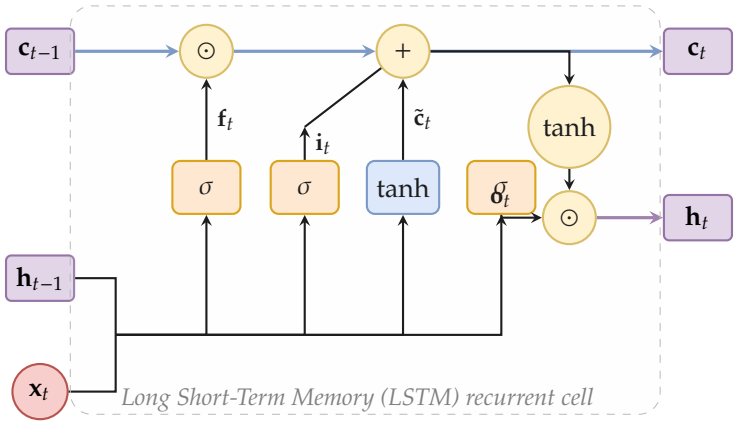


Figure 1.4: Detailed architectural schematic of the Long Short-Term Memory (LSTM) cell, illustrating the linear cell-state trunk ($\mathbf{c}_t$), the multiplicative forget gate ($\mathbf{f}_t$), input gate ($\mathbf{i}_t$), candidate state ($\tilde{\mathbf{c}}_t$), and output gate ($\mathbf{o}_t$).

The interaction between cell memory and incoming data is governed by three non-linear gates, parameterized using logistic sigmoid functions $\sigma(z) = (1 + e^{-z})^{-1} \in [0, 1]$. Because the sigmoid output ranges strictly between 0 and 1, gate activations act as coordinate-wise attenuation coefficients: a value of 0 blocks information transmission entirely, while a value of 1 permits complete transmission.

The algebraic execution of the LSTM unfolds in five sequential stages:

1. **Forget gate**: Determines what fraction of the antecedent cell state $\mathbf{c}_{t-1}$ should be discarded:

$$\mathbf{f}_t = \sigma\left(\mathbf{W}_f \left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_f\right) \in [0, 1]^{d_h}, \tag{1.36}$$

where $[\mathbf{h}_{t-1}, \mathbf{x}_t] \in \mathbb{R}^{d_h + d_x}$ denotes vector concatenation, $\mathbf{W}_f \in \mathbb{R}^{d_h \times (d_h + d_x)}$ is the forget weight matrix, and $\mathbf{b}_f \in \mathbb{R}^{d_h}$ is the forget bias.

2. **Input gate and candidate memory**: Regulates what new information should be added to the persistent cell state:

$$\mathbf{i}_t = \sigma\left(\mathbf{W}_i\left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_i\right) \in [0,1]^{d_h}, \tag{1.37}$$

$$\tilde{\mathbf{c}}_t = \tanh\left(\mathbf{W}_c\left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_c\right) \in (-1,1)^{d_h}, \tag{1.38}$$

where $\mathbf{i}_t$ controls write access, and $\tilde{\mathbf{c}}_t$ creates a new candidate vector of bounded additions.

3. **Cell state update (The Constant Error Carousel)**: The new cell state $\mathbf{c}_t$ is computed via linear combination:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \tag{1.39}$$

where $\odot$ denotes element-wise (Hadamard) vector multiplication.

4. **Output gate and hidden state**: Computes the visible output state by filtering the updated cell state:

$$\mathbf{o}_t = \sigma\left(\mathbf{W}_o\left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_o\right) \in [0,1]^{d_h}, \tag{1.40}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh\left(\mathbf{c}_t\right) \in (-1,1)^{d_h}. \tag{1.41}$$

Proof of the Constant Error Carousel (CEC). To understand why the LSTM eliminates the vanishing gradient pathology, we differentiate the cell state update (Equation 1.39) with respect to the antecedent cell state $\mathbf{c}_{t-1}$:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \operatorname{diag}\left(\mathbf{f}_t\right) + \frac{\partial \mathbf{f}_t}{\partial \mathbf{c}_{t-1}} \odot \mathbf{c}_{t-1} + \frac{\partial \mathbf{i}_t}{\partial \mathbf{c}_{t-1}} \odot \tilde{\mathbf{c}}_t + \mathbf{i}_t \odot \frac{\partial \tilde{\mathbf{c}}_t}{\partial \mathbf{c}_{t-1}}. \tag{1.42}$$

Along the primary diagonal path, the gradient is dominated by the direct linear transmission term:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \approx \operatorname{diag}\left(\mathbf{f}_t\right). \tag{1.43}$$

Across an extended temporal interval from step k to step t, the accumulated error gradient evaluates to:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_k} = \prod_{j=k+1}^{t} \operatorname{diag}\left(\mathbf{f}_j\right). \tag{1.44}$$

Crucially, Equation 1.44 does not involve repeated multiplications of a weight matrix $\mathbf{W}_{hh}^{\top}$, nor does it involve saturating activation derivatives $\tanh'$. If the network learns to saturate the forget gates near one ($\mathbf{f}_j \approx \mathbf{1}$), the gradient product evaluates to the identity matrix:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_k} = \prod_{j=k+1}^{t} \mathbf{I} = \mathbf{I}. \tag{1.45}$$

Under this condition, error gradients propagate backward across arbitrary temporal distances without exponential attenuation, creating a constant error highway (the Constant Error Carousel) that enables the acquisition of long-range dependencies spanning hundreds of steps.

The Gated Recurrent Unit (GRU). In 2014, Cho et al. introduced the Gated Recurrent Unit [**?**], offering a streamlined alternative to the LSTM that merges cell state and hidden state into a single representation $\mathbf{h}_t$, controlled by two gates:

$$\text{Reset gate:} \quad \mathbf{r}_t = \sigma\left(\mathbf{W}_r\left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_r\right) \in [0,1]^{d_h}, \tag{1.46}$$

$$\text{Update gate:} \quad \mathbf{z}_t = \sigma\left(\mathbf{W}_z\left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_z\right) \in [0,1]^{d_h}, \tag{1.47}$$

$$\text{Candidate state:} \quad \tilde{\mathbf{h}}_t = \tanh\left(\mathbf{W}_h\left[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_h\right) \in (-1,1)^{d_h}, \tag{1.48}$$

$$\text{State interpolation:} \quad \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \tag{1.49}$$

The update gate $\mathbf{z}_t$ acts as an adaptive convex interpolator between past memory $\mathbf{h}_{t-1}$ and candidate update $\tilde{\mathbf{h}}_t$, performing the combined roles of the LSTM's forget and input gates while reducing parameter count by approximately 25%.

Architectural Property	**Vanilla Elman RNN**	**LSTM**	**GRU**
Separate cell memory ($\mathbf{c}_t$)	No	Yes	No
Number of gating sub-units	0	3 ($\mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t$)	2 ($\mathbf{r}_t, \mathbf{z}_t$)
Parameter count scaling	$O\left(d_h^2 + d_h d_x\right)$	$O\left(4(d_h^2 + d_h d_x)\right)$	$O\left(3(d_h^2 + d_h d_x)\right)$
Gradient propagation highway	None (exponential decay)	Linear additive trunk	Linear convex interpolation
Computational parallelizability	Strictly serialized	Strictly serialized	Strictly serialized

Table 1.3: Architectural comparison between the vanilla Elman RNN, the Long Short-Term Memory (LSTM) network, and the Gated Recurrent Unit (GRU).

As summarized in Table 1.3, while gated architectures resolve the analytical vanishing gradient problem through additive linear memory trunks, they remain fundamentally serialized along the sequence dimension, preserving the computational bottleneck that prevents efficient parallel scaling.

1.5 Sequence-to-sequence modeling, encoder-decoder architectures, and the information bottleneck

All recurrent configurations examined thus far operate under a *one-to-one mapping paradigm*, where each incoming token $\mathbf{x}_t$ immediately triggers the emission of a corresponding prediction $\hat{\mathbf{y}}_t$. While suitable for synchronous sequence tagging, this formulation fails when applied to general sequence-to-sequence transformations.

1.5.1 Limitations of one-to-one sequence mapping

Synchronous one-to-one sequence processing exhibits two fatal structural failure modes when applied to complex linguistic tasks such as machine translation, summarization, or dialogue:

1. **Sequence length disparity ($T_x \neq T_y$)**: In natural language translation, source and target sentences rarely contain the same number of words. Consider translating an English utterance into Italian:

$$\text{English source:} \quad \texttt{[It] [is] [indeed] [sunny] [today]} \quad (T_x = 5 \text{ tokens}), \tag{1.50}$$

$$\text{Italian target:} \quad \texttt{[Effettivamente] [oggi] [è] [soleggiato]} \quad (T_y = 4 \text{ tokens}). \tag{1.51}$$

   Under a one-to-one mapping architecture, emitting 4 Italian words requires ingesting exactly 4 English tokens, leaving the 5th token unaligned. If the target sequence is longer than the source ($T_y > T_x$), the recurrent unit exhausts its input tokens and has no valid data to ingest for subsequent steps.

2. **Non-monotonic alignment and lack of future context**: Even if sequence lengths happen to match, the semantic alignment between languages is non-monotonic and structurally inverted. In the example above, the initial Italian word `"Effettivamente"` translates the English adverb `"indeed"`, which occupies the *third* position in the source sentence. Under causal one-to-one processing, the network must generate the first output token after observing only `"It"`, long before the conditioning token `"indeed"` has entered the model.

Table 1.4 illustrates this structural misalignment, demonstrating why causal synchronous emission cannot resolve cross-lingual translation.

Step t	Source Token $\mathbf{x}_t$	Target Token y_t^*	Semantic Alignment	Feasible Under Causal One-to-One?
1	`It`	`Effettivamente`	Maps to $\mathbf{x}_3$ (`indeed`)	**No** (future context required)
2	`is`	`oggi`	Maps to $\mathbf{x}_5$ (`today`)	**No** (future context required)
3	`indeed`	`è`	Maps to $\mathbf{x}_2$ (`is`)	**No** (inverted syntax)
4	`sunny`	`soleggiato`	Maps to $\mathbf{x}_4$ (`sunny`)	Partially (local match)
5	`today`	$\varnothing$	No target token	**No** (length mismatch)

Table 1.4: Cross-lingual sequence alignment between English source and Italian target sentences, illustrating the failure of causal one-to-one recurrent mapping due to non-monotonic word order and length disparity.

1.5.2 The encoder-decoder paradigm

To resolve the constraints of one-to-one mapping, Sutskever et al. [**?**] and Cho et al. [**?**] decoupled sequence processing into two specialized recurrent networks: an *Encoder* and a *Decoder*.

The operational workflow of the Seq2Seq architecture (Figure 1.5) decouples sequence encoding from generation through three distinct phases:

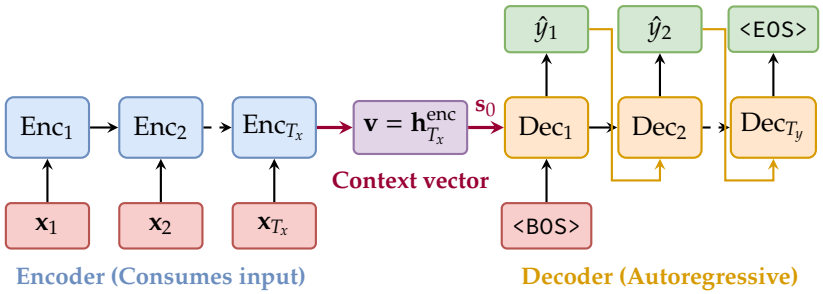


Figure 1.5: The Sequence-to-Sequence (Seq2Seq) Encoder-Decoder architecture. The Encoder ingests the full source sequence of length T_x, compressing it into a single context vector $\mathbf{v}$. The Decoder is initialized with $\mathbf{v}$ and generates target tokens autoregressively until emitting the end-of-sequence delimiter <EOS>.

1. **Sequence encoding**: The Encoder RNN sequentially ingests the entire source sequence $\mathbf{x}_1, \ldots, \mathbf{x}_{T_x}$ without generating any output tokens. At the final source token T_x, the encoder's terminal hidden state is extracted to serve as the *context vector*:
$$\mathbf{v} = \mathbf{h}^{\text{enc}}_{T_x} \in \mathbb{R}^{d_h}. \tag{1.52}$$
The vector $\mathbf{v}$ represents a compressed semantic summary of the entire source utterance.

2. **Decoder state initialization**: The Decoder RNN initializes its internal state using the context vector:
$$\mathbf{s}_0 = \mathbf{v}, \tag{1.53}$$
thereby conditioning all subsequent generation steps on the global context of the input.

3. **Autoregressive generation**: Generation commences by feeding a synthetic start-of-sequence token, denoted <BOS> (Beginning of Sequence), into the decoder:
$$\mathbf{s}_1 = \text{DecoderRNN}\left(\mathbf{s}_0, \mathbf{e}_{\texttt{<BOS>}}\right). \tag{1.54}$$
The decoder projects state $\mathbf{s}_1$ to emit a categorical probability distribution over target vocabulary $\mathcal{V}_y$, sampling or greedily selecting token $\hat{y}_1$. In the subsequent step, the previously generated token $\hat{y}_1$ is fed back as the input token to compute $\mathbf{s}_2$:
$$\mathbf{s}_t = \text{DecoderRNN}\left(\mathbf{s}_{t-1}, \mathbf{e}_{\hat{y}_{t-1}}\right). \tag{1.55}$$
Generation terminates when the decoder emits a dedicated end-of-sequence token, denoted <EOS> (End of Sequence), allowing target length T_y to be determined dynamically by the model.

Mathematically, the encoder-decoder framework models the conditional probability of target sequence $Y = (y_1, \ldots, y_{T_y})$ given source sequence $X = (\mathbf{x}_1, \ldots, \mathbf{x}_{T_x})$ via the exact probability chain rule:
$$P(Y \mid X) = \prod_{t=1}^{T_y} P\left(y_t \mid y_1, \ldots, y_{t-1}, \mathbf{v}\right). \tag{1.56}$$

1.5.3 Empirical validation: revised Sherlock Holmes casing

To evaluate how the encoder-decoder paradigm resolves the causal horizon failure identified in Section 1.3.3, we revisit the character casing benchmark on the ambiguous test utterance:

$$\text{Input text:} \quad \texttt{"indeed iam sherlock holmes,"} \tag{1.57}$$
$$\text{Target text:} \quad \texttt{"Indeed I am Sherlock Holmes,"} \tag{1.58}$$

Under the one-to-one causal RNN evaluated in Section 1.3.3, the model committed four distinct capitalization errors:

1. **The initial token `'i'` in `indeed`**: Capitalized correctly only because it immediately followed an opening quote.
2. **The isolated pronoun `'i'` in `iam`**: Remained lowercase (`'i'`) because the causal model, seeing only antecedent tokens `"indeed "`, could not determine whether `'i'` represented the personal pronoun *I* or the initial letter of common words such as *in*, *is*, *it*, *if*.
3. **The initial `'s'` in `sherlock`**: Remained lowercase because `"iam "` is followed by lowercase words in over 90% of training contexts.
4. **The initial `'h'` in `holmes`**: Capitalized correctly only through local bigram memorization following `sherlock`.

When the exact same sequence is evaluated using a Sequence-to-Sequence Encoder-Decoder architecture, the model resolves all four tokens without ambiguity:

$$\text{Seq2Seq Output:} \quad \texttt{"Indeed I am Sherlock Holmes,"} \tag{1.59}$$

The conceptual basis for this success resides in the timing of representation formation. In the Seq2Seq paradigm, the Encoder ingests the entire sentence `"indeed iam sherlock holmes,"` before the Decoder generates a single character. When the Decoder initiates generation from $\mathbf{s}_0 = \mathbf{v}$, the context vector $\mathbf{v}$ already encodes the presence of the proper noun `sherlock holmes` and the pronoun `i`. With full bidirectional information synthesized into the context vector, the decoder generates correctly capitalized characters from the very first token, resolving ambiguities that are mathematically unsolvable under causal one-to-one processing.

1.5.4 The information bottleneck and the prelude to attention

Despite the empirical triumph of the Encoder-Decoder paradigm on short sentences, the architecture encounters an inescapable theoretical barrier: the *fixed-vector information bottleneck* [**?**].

In the standard Seq2Seq formulation, the Encoder must compress all semantic, syntactic, stylistic, and relational information present in a source sequence of arbitrary length T_x into a single, fixed-dimensional Euclidean vector $\mathbf{v} \in \mathbb{R}^{d_h}$. Mathematically, this transformation represents a severe dimensionality reduction mapping:

$$f_{\text{enc}} : \mathbb{R}^{T_x \times d_x} \longrightarrow \mathbb{R}^{d_h}. \tag{1.60}$$

For a short sentence of 5 words, compressing the input into $d_h = 512$ dimensions preserves sufficient fidelity for accurate reconstruction. However, as sequence length increases to 20, 50, or 100 words, the amount of information in the source sequence grows linearly with T_x, while the capacity of the context vector $\mathbf{v}$ remains strictly constant at d_h.

Consequently, the context vector becomes saturated. As the encoder processes later words in a long document, earlier clauses are progressively overwritten and attenuated by the recurrent transition operator. When the decoder begins generating target tokens, the context vector $\mathbf{v}$ retains vivid representations of the final few source tokens, but retains only degraded, blurred traces of earlier tokens.

Empirical evaluations by Cho et al. [**?**] and Bahdanau et al. [**?**] demonstrated that the translation performance of standard RNN encoder-decoder models plummets catastrophically when sentence length exceeds 15 to 20 words. Even when employing multi-layer LSTMs with thousands of hidden units, compressing an arbitrary-length document into a single static vector remains an insurmountable information-theoretic bottleneck.

We are therefore confronted with the dual architectural crises of recurrent sequence modeling:

1. **The computational crisis**: The strict sequential recurrence $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$ serializes training along the sequence dimension, precluding GPU hardware parallelization and enforcing an $O(T)$ training wall-clock runtime.

2. **The representational crisis**: Compressing variable-length context into a single static state vector $\mathbf{v} \in \mathbb{R}^{d_h}$ destroys long-range dependencies and forces decoders to reconstruct sentences from lossy, blurred summaries.

Resolving these dual crises requires a fundamental paradigm shift: replacing static vector compression with dynamic, query-conditioned associative retrieval. Rather than forcing a single vector to summarize an entire sequence, the model should maintain a collection of all intermediate encoder states and allow the decoder to dynamically "attend" to the most relevant source positions at each step of generation. This foundational breakthrough—the Attention Mechanism—forms the conceptual bridge that dismantles the recurrent paradigm and underpins the modern Transformer architecture.